

TÌM HIỂU CƠ SỞ LÝ THUYẾT BÀI TẬP 2: N8N

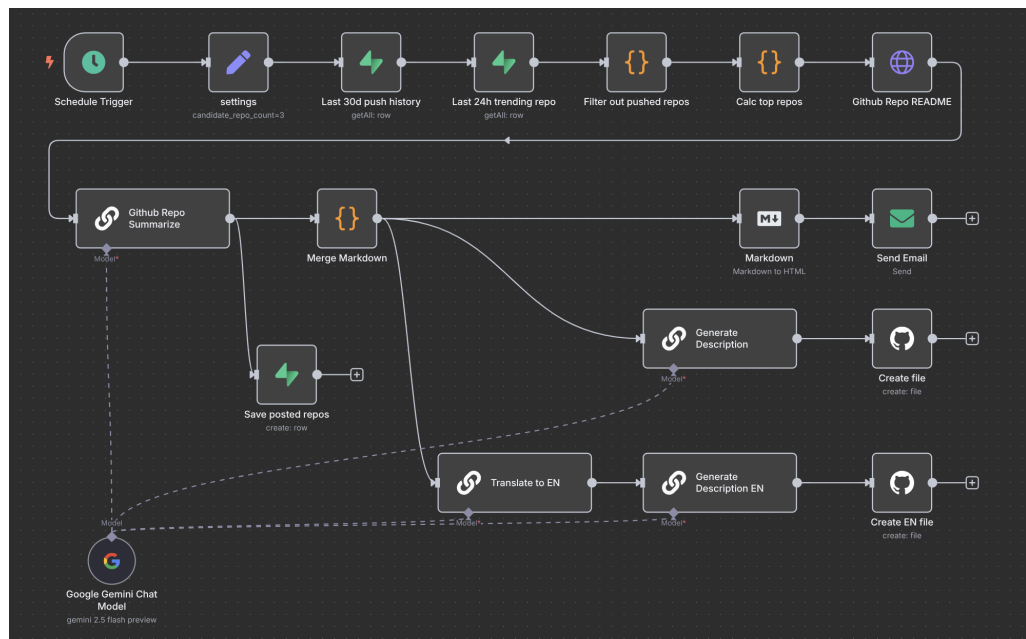
Mục lục

1. N8N.....	2
1.1. Tổng quan về n8n.....	2
1.2. Cài đặt n8n.....	2
1.3. Những khái niệm cơ bản trong n8n.....	3
1.4. Kiến trúc và cách n8n hoạt động.....	5
1.4.1. Kiến trúc.....	5
1.4.2. Cách hoạt động.....	6

1. N8N

1.1. Tổng quan về n8n

- n8n là một nền tảng tự động hóa mã nguồn mở (theo hướng drag & drop các node) để xây dựng nhanh luồng hoạt động, làm điểm điều hướng kết nối các ứng dụng với nhau để tăng tốc và cải tiến quá trình làm việc.
- Điểm đặc biệt của n8n chính là mã nguồn mở, miễn phí và công khai, ngoài ra còn cho phép Self-hosted (tự host, không bị phụ thuộc vào cloud của n8n → Có thể chọn host qua docker, railway,... tùy ý). Ngoài ra giao diện drag & drop linh hoạt, kết nối các node với nhau tạo thành luồng hoạt động một cách trực quan, dễ theo dõi, kiểm soát và sửa đổi cũng như tích hợp chatbot một cách đơn giản.
- n8n xài bản web thì nó sẽ nhảy ra cloud, cloud thì có tính phí. Nên chuyển thành self-hosted thì tùy ý tính chỉnh (tương tự như cách wordpress hoạt động, tức là clone về, chạy local)



1.2. Cài đặt n8n

Tìm hiểu tại đây: [github n8n](#)

- Có 2 cách cài đặt chính: sử dụng docker hoặc qua npx/npm (cần [node.JS](#)). (Lưu ý: npx là lệnh chạy thực thi, chạy một lần không xài lâu dài, npm là tải về và lưu trữ, có thể dùng lại sau)

→ Nên cài docker để đỡ quản lí cũng như dọn enviroment đơn giản và thấy cũng yêu cầu sử dụng docker) (Nếu tải bằng npm về rồi dockerize sau thì tính tùy biến cao, có thể tích hợp thêm những gì muốn bỏ lên nhưng trong khuôn khổ bài tập thì không cần thiết)

Quick Start

Try n8n instantly with [npx](#) (requires [Node.js](#)):

```
npx n8n
```

Or deploy with [Docker](#):

```
docker volume create n8n_data
docker run -it --rm --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

Access the editor at <http://localhost:5678>

Hướng dẫn cài đặt n8n bằng docker, sử dụng db postgres để lưu trữ dữ liệu:

Bước	Hành động	Lệnh thực thi trong CMD (Chép từng dòng một)	Giải thích
1	Tạo mạng kết nối	<code>docker network create n8n-network</code>	Tạo một "đường truyền riêng" để n8n, Postgres và Ngrok thấy nhau.
2	Tạo ổ cứng n8n	<code>docker volume create n8n_data</code>	(Nếu đã tạo rồi thì bỏ qua bước này).
3	Tạo ổ cứng DB	<code>docker volume create postgres_data</code>	(Nếu đã tạo rồi thì bỏ qua bước này).

4	Chạy Postgres	<pre>docker run -d --name postgres-n8n --network n8n-network -p 5432:5432 -v postgres_data:/var/lib/postgresql/d ata -e POSTGRES_USER=n8n_user -e POSTGRES_PASSWORD=n8n_p assword -e POSTGRES_DB=n8n_db postgres:16-alpine</pre>	Chạy database trong mạng n8n-network. Dữ liệu sẽ nằm trong volume postgres_data.
5	Chạy Ngrok	<pre>docker run -d --name ngrok-n8n --network n8n-network -e NGROK_AUTHTOKEN=TOKE N_HERE ngrok/ngrok:latest http --domain=cousinly-unlogistical-ke ven.ngrok-free.dev n8n:5678</pre>	Vào đây lấy token : https://dashboard.ngrok.com/get-started/your-authtoken (Lưu ý: đừng config hay paste lung tung làm lộ key)
5	Chạy n8n	<pre>docker run -d --name n8n --network n8n-network -p 5678:5678 -e N8N_HOST=cousinly-unlogistical -keven.ngrok-free.dev -e N8N_PROTOCOL=https -e WEBHOOK_URL=https://cousinl y-unlogistical-keven.ngrok-free.de v/ -e DB_TYPE=postgresdb -e DB_POSTGRESDB_HOST=post gres-n8n -e DB_POSTGRESDB_PORT=5432 -e DB_POSTGRESDB_DATABASE =n8n_db -e DB_POSTGRESDB_USER=n8n_ user -e DB_POSTGRESDB_PASSWOR D=n8n_password -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n</pre>	Kết nối n8n vào cùng mạng. Host lúc này chỉ cần ghi là postgres-n8n.

1.3. Những khái niệm cơ bản trong n8n

Khái niệm	Định nghĩa	Chức năng chính	Ví dụ minh họa
Workflow	Bản đồ tổng thể của quy trình.	Kết nối các Node để tạo thành một chuỗi xử lý tự động hoàn chỉnh.	Quy trình tự động lưu đính kèm từ Gmail vào Google Drive.
Node	Đơn vị xử lý nhỏ nhất (Nút).	Thực hiện một nhiệm vụ cụ thể (Kích hoạt, Xử lý, hoặc Gửi dữ liệu).	Node Gmail , Node Set , Node IF .
Trigger	Điểm khởi đầu của Workflow.	Theo dõi sự kiện và tự động chạy Workflow khi điều kiện thỏa mãn.	Chạy khi có Webhook gửi đến hoặc chạy theo Lịch trình (Cron) .
Connection	Đường nối giữa các Node.	Xác định hướng đi của dữ liệu từ bước này sang bước tiếp theo.	Mũi tên nối từ Node A sang Node B.
Item	Đơn vị dữ liệu (đối tượng).	n8n xử lý dữ liệu theo danh sách các Item (mỗi Item là một bản ghi).	1 dòng trong Excel = 1 Item; 1 tin nhắn = 1 Item.
Expression	Biểu thức/Công thức.	Lấy dữ liệu động từ các Node trước đó hoặc tính toán toán học/logic.	<code>{{ \$json.email }}</code> để lấy địa chỉ email khách hàng một cách tự động.
Credentials	Thông tin xác thực.	Lưu trữ an toàn API Key, Token hoặc mật khẩu của các ứng dụng ngoại vi.	Đăng nhập tài khoản Google hoặc Slack để n8n có quyền truy cập.

Execution	Lịch sử thực thi.	Lưu lại chi tiết mỗi lần Workflow chạy (thành công hay thất bại).	Kiểm tra xem tại sao đơn hàng số #102 không gửi được email.
Input/Output	Đầu vào và Đầu ra.	Cổng nhận và xuất dữ liệu của mỗi Node.	Node "Filter" nhận 10 Items (Input) nhưng chỉ xuất ra 3 Items thỏa mãn (Output).
Environment Variables	Biến môi trường.	Lưu các giá trị dùng chung cho toàn bộ hệ thống (URL, ID cố định).	Lưu <code>DOMAIN_NAME</code> để dùng cho tất cả các Workflow khác nhau.

1.4. Kiến trúc và cách n8n hoạt động

1.4.1. Kiến trúc

Kiến trúc của n8n được xây dựng theo mô hình node-based, có thể hiểu theo ba tầng chính: cấu trúc bên trong (core), mô hình triển khai (deploy) và cơ chế xử lý dữ liệu.

Các thành phần lõi:

Thành phần	Vai trò	Chi tiết kỹ thuật
Editor UI	Giao diện người dùng	Chạy trên trình duyệt (Vue.js), giúp người dùng kéo thả các Node và cấu hình Workflow.
Server (Backend)	Bộ não điều khiển	Xây dựng trên Node.js và TypeScript . Xử lý các yêu cầu từ UI và quản lý việc chạy Workflow.
Workflow Engine	Bộ máy thực thi	Chịu trách nhiệm đọc sơ đồ Workflow, kích hoạt các Node theo thứ tự và truyền dữ liệu giữa chúng.
Database	Kho lưu trữ	Lưu trữ cấu hình Workflow, Credentials và lịch sử Execution. Hỗ trợ SQLite (mặc định), PostgreSQL hoặc MySQL.

- **n8n editor (frontend / UI):**
 - Là giao diện chính kéo thả của n8n cho user
 - Là một cái web application build bằng Vue và JS
 - Mỗi khi tạo một cái workflow thì editor nó sẽ tạo ra một cái JSON đại diện cho cái workflow đó
- **Trình thực thi Workflow (backend/worker):**
 - Đây là cốt lõi nơi thực thi những workflow đã xây dựng
 - Lấy cái JSON của editor đã làm để chạy theo thứ tự
 - Xử lý khởi chạy từng node và logic luận lý, dữ liệu, xử lý lỗi và tạo log
 - Để có thể làm tăng tính co giãn của một workflow trong n8n thì có thể xây dựng một cái workflow chính rồi sau đó gắn thêm những workflow con vào → tăng tính mở rộng, dễ tinh chỉnh sửa chữa, tránh phải sửa quá nhiều node khi thay đổi một luồng hoạt động
- **Node (nút):**
 - Trigger node: trigger node là một node đặc biệt dùng để bắt đầu workflow, có thể kích hoạt bằng webhooks, schedule (cron), manual trigger hoặc là một sự kiện của ứng dụng bên thứ ba
 - Regular node: một node bình thường thực hiện một chức năng cố định như đọc dữ liệu từ database, gửi mail, gọi API,...
 - Có thể hiểu đơn giản mỗi node giống như một hàm có tham số nhận vào và có dữ liệu trả về. Mỗi node hoàn toàn hoạt động độc lập với nhau, không bị phụ thuộc.
- **Database (cơ sở dữ liệu):**
 - n8n có sử dụng cơ sở dữ liệu (SQLite là mặc định của nó, ngoài ra nó hỗ trợ SQL Server, My SQL hoặc Maria DB) để lưu trữ: định nghĩa workflow, kết nối và credentials, lịch sử,... (cách hoạt động này tương tự với wordpress, tức là lưu trữ chính là lưu cấu hình và trạng thái)

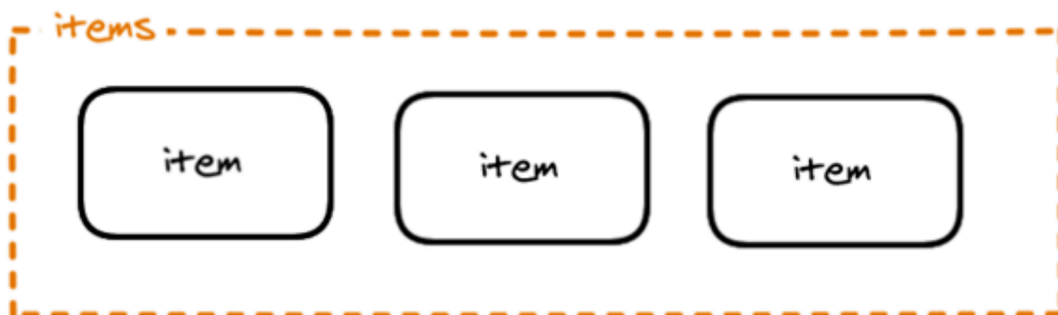
1.4.2. Cách hoạt động

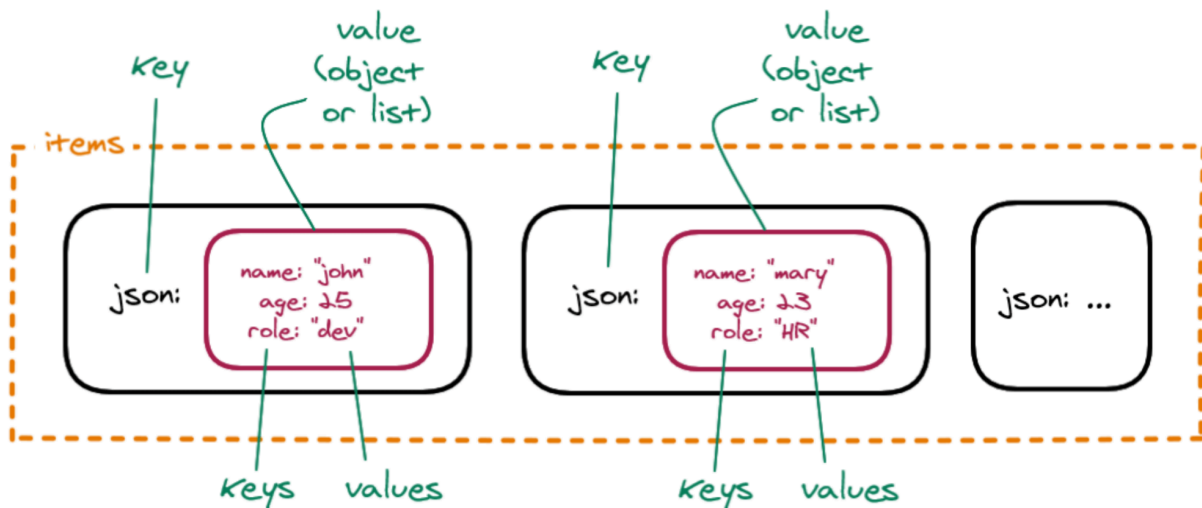
Xem tài liệu giải thích tại đây: [Link](#)

- n8n hoạt động tuần tự từ node kích hoạt (trigger node) → qua các nhánh logic và thực thi các node khác, đem dữ liệu đi lần lượt theo logic workflow.



- Dữ liệu được chuyển đi đều được định dạng dưới dạng một mảng (array) là danh sách chứa JSON object, mà mỗi object được xem như là một item
 - Cấu trúc chuẩn của dữ liệu: [{ "key": "value" }, { "key": "value" }]





Ví dụ về dữ liệu được gửi đi được cấu hình trong một node:

```
1 // Định dạng thông tin commit để hiển thị trong Slack
2 return items.map(item => {
3   const commit = item.commit;
4   const repo = item.repository;
5
6   return {
7     json: {
8       commitInfo: {
9         message: commit.message,
10        author: commit.author.name,
11        date: commit.author.date,
12        url: item.html_url,
13        repo: `${repo.owner.login}/${repo.name}`,
14        branch: item.parents[0] ? item.parents[0].sha.substring(0, 7) : 'unknown'
15      }
16    }
17  };
18 });
```

- Khi danh sách được truyền đi, mỗi node sẽ tự động xác định số lượng item → lặp qua toàn bộ các items có trong danh sách dữ liệu. 100 items? → Lặp 100 lần tại node đó qua mỗi item.

Customer Datastore (n8n training)

Execute node

Parameters Docs

Operation: Get All People

Return All: ☒

Total items returned by the node: 5 items

id	name	email	notes	country	created
23423532	Jay Gatsby	gatsby@west-egg.com	Keeps asking about a green light??	US	1925-04-10
23423533	José Arcadio Buendía	jab@macondo.co	Lots of people named after him. Very confusing	CO	1967-05-05
23423534	Max Sendak	info@in-and-out-of-weeks.org	Keeps rolling his terrible eyes	US	1963-04-09
23423535	Zaphod Beeblebrox	captain@heartofgold.com	Felt like I was talking to more than one person	[null]	1979-10-12
23423536	Edmund Pevensie	edmund@narnia.gov	Passionate sailor	UK	1950-10-16

Item 1

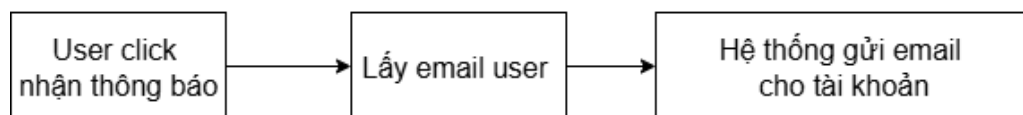
Item 2

[NGOÀI LỀ - Thắc mắc tự tìm thêm]

- 1) Vậy dữ liệu được chuyển tải qua các node là array, có index, nó có ích lợi gì không, có gì hay cần hiểu không ?

Ví dụ hệ thống có 3 node, node đầu kích hoạt nhận sự kiện click của user, node thứ hai lấy email của user, node thứ ba hệ thống gửi mail xác nhận cho user và có hơn 100 click cùng một lúc.

Trigger Node



Bộ dữ liệu được gửi đi như sau:

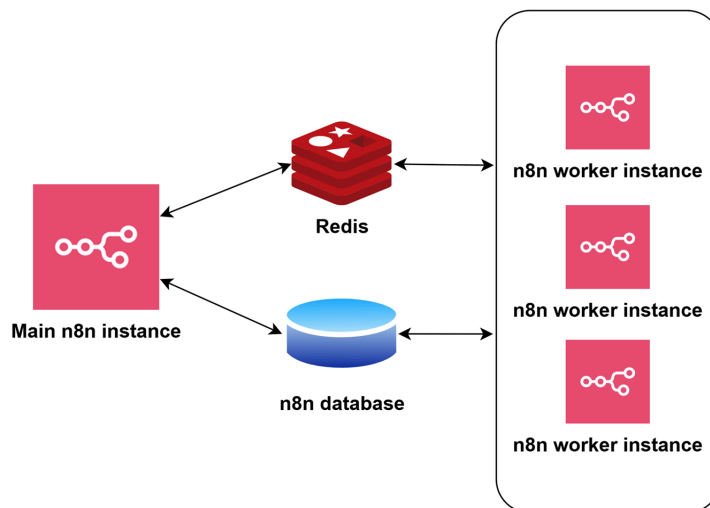
```

[
  { "name": "An", "email": "an@gmail.com" }, // Item 0
  { "name": "Bình", "email": "binh@gmail.com" }, // Item 1
  { "name": "Chi", "email": "chi@gmail.com" } // Item 2
  ... // các users khác
]
  
```

Khi nó chuyển từ node 1 → node 2 thì node 2 sẽ hiểu là có tổng cộng 100 công việc cần làm (tương ứng với 100 items) → Tức là node 2 sẽ tự động chạy 100 lần với các item theo thứ tự.

Vậy thì việc chuyển đi thành mảng có gì tốt ?

- **Vòng lặp:** Giúp cho việc looping được nhanh hơn, chẳng hạn như index của item đến 99 → lặp 100 lần mà không cần phải khởi tạo node riêng để xử lý logic tính toán vòng lặp. (khác với nền tảng như Zapier, 1 click tính là một lần chạy qua toàn bộ vòng lặp riêng biệt → 100 luồng, nếu muốn xử lý looping thì phải cấu hình. Còn n8n thì xử lý một lần nhiều dữ liệu → 1 luồng → giảm thời gian và chi phí vận hành, giảm thiểu cấu hình)
- **Truy vấn ngược:** ở node 1 có “name”, ở node 2 có “email”, việc thực thi gửi mail ở node 3 có thể truy vấn trở lại nếu có lỗi xảy ra thì đưa nào là đưa năm dữ liệu gốc được gửi đi. Ví dụ lỗi ở item 2 tại node 3 → xác định vấn đề là email không hợp lệ → tiến hành truy vấn ngược lại với index = 2 ở các node trước để xác định dữ liệu gốc đang ở node 2 (có phải ghép regex lỗi? kéo dữ liệu lỗi?)
- **Xử lý đồng thời:** Quan trọng là tính năng xử lý song song, việc item có index giúp hệ thống có thể chia việc cho nhiều “worker” để xử lý đồng thời, đánh index giúp nhận biết chia task từ đâu đến đâu cho các worker đó. Giảm tải cho main instance.



2) Đã nhắc về “Main instance” và “Worker” trong n8n, vậy chúng là như thế nào?

- Main instance và worker là khái niệm xuất hiện khi xây dựng workflow theo hướng dễ mở rộng, áp dụng hàng đợi (queue) vào việc xử lý request. Ý tưởng là chia ra cho “Main Instance” là người order đẩy task vào hàng đợi rồi để các “Worker” xử lý từng đơn.
→ Càng nhiều worker, càng giảm tải cho main instance, xử lý nhanh hơn vì có nhiều người cùng làm.
- Main instance và worker bản chất đều sử dụng chung một workflow, nhưng main instance có giao diện còn workflow chỉ là những trình chạy riêng (vd một container docker). Ngoài ra quan trọng là khác nhau ở lệnh khởi động:
 - Với main instance, lệnh khởi động là: “n8n”
 - Với worker, lệnh khởi động là: “n8n worker”

3) Nếu “Main instance” và “Worker” đều có chung một workflow thì làm sao để biết khi nào main instance xử lý, khi nào worker xử lý, khi nào thì worker được kích hoạt?

Quy trình là như sau:

Trigger Event → Main instance bắt sự kiện → Main instance tạo phiếu công việc có id riêng biệt thành các job → Đẩy vào hàng đợi trong redis → Worker xem hàng đợi các job chưa được xử lý → Xem id của job → Tìm lại workflow và dữ liệu trong db chung → Dem ra xử lý

Và để sử dụng worker thì phải cấu hình qua chế độ “queue mode” trong n8n để xác thực rằng worker được phép chạy. Còn ở chế độ mặc định thì main instance làm hết.

Khi chế độ “queue mode” được áp dụng thì main instance bị ép buộc vào thế chỉ được phép đẩy job vào redis để worker làm.

4) Vậy nếu chỉ có 1 main instance và 1 worker thì nó có lãng phí tài nguyên không khi không mở rộng ra nhiều worker khác?

- Về mặt tài nguyên, đúng là có lãng phí về hiệu suất khi tạo dư nếu nhu cầu không đến (1 sếp? 1 lính?) (5-10 items?)
- Nhưng về mặt quản lý rủi ro thì việc áp dụng hình thức này có những ưu điểm riêng:
 - **Cô lập rủi ro:** nếu không có worker, ví dụ như xử lý 1000 items siêu cấp nặng → ngốn RAM và CPU. Nếu chỉ có một mình main instance đang chạy thì nó sẽ bị treo, mà main instance thì bao gồm giao diện → đứng giao diện, không thể thao tác theo dõi cũng không thể nhấn dừng lại.
 - **Không làm mất sự kiện hay giảm tốc độ phản hồi:** Ví dụ như chỉ có 1 mình main instance đang chạy mà bị crash hay lỗi thì có thể kéo theo làm drop các sự kiện theo sau vì đang xử lý một khối lượng việc quá lớn, phản hồi webhooks cũng sẽ bị delay → Để worker chạy thì nó kéo job trên redis, có lỗi có crash thì

job cũng đang chờ ở redis, main instance vẫn nhận sự kiện mới bình thường rồi đẩy vào redis xử lý sau.